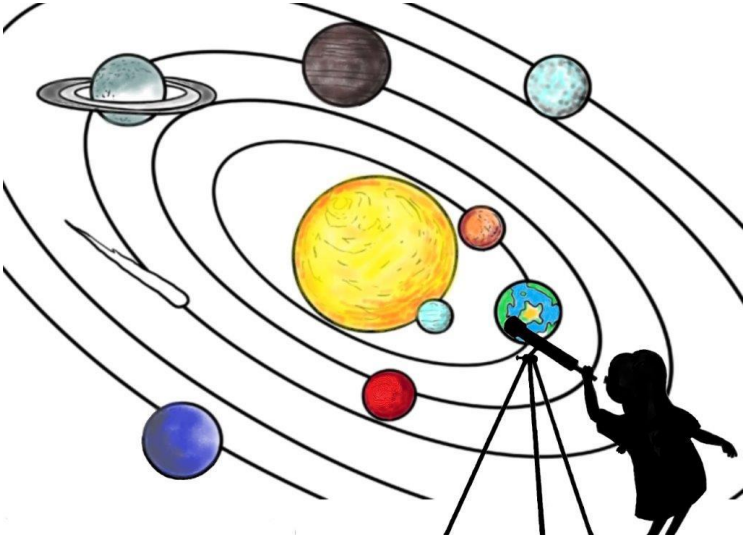
4.1 Concepts



Cippi studies the stars

To appreciate the impact of concepts to its full extent, I want to start with a short motivation for concepts.

4.1.1 Two Wrong Approaches

Prior to C++20, we had two diametrically opposed ways to think about functions or classes: defining them for specific types, or defining them for generic types. In the latter case, we call them function templates or class templates. Both approaches have their own set of problems:

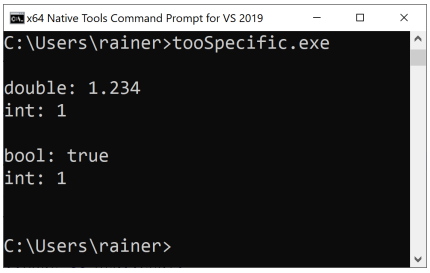
4.1.1.1 Too Specific

It's tedious work to overload a function or reimplement a class for each type. To avoid that burden, type conversion often comes to our rescue. What seems like a rescue is often a curse.

Implicit conversions

```
1  // tooSpecific.cpp
2
3  #include <iostream>
4
5  void needInt(int i){
6      std::cout << "int: " << i << '\n';
7  }
8
9  int main(){
10
11     std::cout << std::boolalpha << '\n';
12
13     double d{1.234};
14     std::cout << "double: " << d << '\n';
15     needInt(d);
16
17     std::cout << '\n';
18
19     bool b{true};
20     std::cout << "bool: " << b << '\n';
21     needInt(b);
22
23     std::cout << '\n';
24
25 }
```

In the first case (line 13), I start with a `double` and end with an `int` (line 15). In the second case, I start with a `bool` (line 19) and also end with an `int` (line 21).



Implicit conversions

The program exemplifies two implicit conversions.

4.1.1.1.1 Narrowing Conversion

Invoking `getInt(int a)` with a `double` gives you narrowing conversion. Narrowing conversion is a conversion, including a loss of accuracy. I assume this is not what you want.

4.1.1.1.2 Integral Promotion

But the other way around is also not better. Invoking `getInt(int a)` with a `bool` promotes the `bool` to an `int`. Surprised? Many C++ developers don't know which data type they get when they add two `bool`s.

Adding two `bool`s

```
template <typename T>
auto add(T first, T second){
    return first + second;
}

int main(){
    add(true, false);
}
```

C++ Insights[1] visualizes the source code above after the compiler transformed the function template in an instantiation.

[1] https://cppinsights.io/s/9bd14f99

```
1 template <typename T>
2 auto add(T first, T second){
3     return first + second;
4 }
5
6 #ifdef INSIGHTS_USE_TEMPLATE
7 template<>
8 int add<bool>(bool first, bool second)
9 {
10   return static_cast<int>(first) + static_cast<int>(second);
11 }
12 #endif
13
14
15 int main()
16 {
17   add(true, false);
18 }
```

bool to int promotion

Lines 6 - 12 are the crucial ones in this screenshot of C++ Insights[2]. The template instantiation of the function template `add` creates a full specialization with the return type `int`. Both `bools` are implicitly promoted to `int`.

My conviction is that we rely for convenience on the magic of conversions, because we don't want to overload a function or reimplement a class for each type.

Let me try the other way and use a generic function. Maybe this is our rescue?

4.1.1.2 Too Generic

Sorting a container is a general idea. It should work for each container if its elements support ordering. In the following example, I apply the standard algorithm `std::sort` to the standard container `std::list`.

[2] https://cppinsights.io/

Sorting a `std::list`

```
// tooGeneric.cpp

#include <algorithm>
#include <list>

int main(){

    std::list<int> myList{1, 10, 3, 2, 5};

    std::sort(myList.begin(), myList.end());

}
```

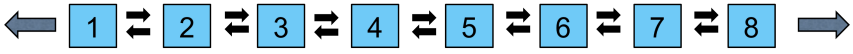
A compiler error when trying to sort a `std::list`

I don't even want to decipher this long message. What's gone wrong? Let's take a

look at the signature of the specific overload of `std::sort`[3] used in this example.

```
template< class RandomIt >
constexpr void sort( RandomIt first, RandomIt last );
```

`std::sort` uses strange-named argument types such as `RandomIt`. `RandomIt` stands for a random-access iterator and gives the key hint for the overwhelming error message. A `std::list` only provides a bidirectional iterator, but `std:sort` requires a random-access iterator. The following graphic shows why a `std::list` does not support a random access iterator.



The structure of a `std::list`

If you study the `std::sort` documentation on cppreference.com, you will find something exciting: type requirements on template parameters. They place conceptual requirements on the types that have been formalized into the C++20 feature, concepts.

4.1.1.3 Concepts to the Rescue

Concepts put semantic constraints on template parameters. `std::sort` has overloads that accept a comparator.

```
template< class RandomIt, class Compare >
constexpr void sort(RandomIt first, RandomIt last, Compare comp);
```

These are the type requirements for the more powerful overload of `std::sort`:

- `RandomIt` must meet the requirements of `ValueSwappable` and `LegacyRandomAccessIterato`
- The type of the dereferenced `RandomIt` must meet the requirements of `MoveAssignable` and `MoveConstructible`.
- The type of the dereferenced `RandomIt` must meet the requirements of `Compare`.

[3] https://en.cppreference.com/w/cpp/algorithm/sort

Requirements such as `ValueSwappable` or `LegacyRandomAccessIterator` are so-called named requirements. Some of these requirements are formalized in C++20 in concepts[4].

In particular, `std::sort` requires a `LegacyRandomAccessIterator`. Let's have a closer look at the named requirement `LegacyRandomAccessIterator` that is called `random_access_iterator` (part of `<iterator>`) in C++20:

`std::random_access_iterator`

```
template<class I>
    concept random_access_iterator =
        bidirectional_iterator<I> &&
        derived_from<ITER_CONCEPT(I), random_access_iterator_tag> &&
        totally_ordered<I> &&
        sized_sentinel_for<I, I> &&
        requires(I i, const I j, const iter_difference_t<I> n) {
            { i += n } -> same_as<I&>;
            { j + n } -> same_as<I>;
            { n + j } -> same_as<I>;
            { i -= n } -> same_as<I&>;
            { j - n } -> same_as<I>;
            { j[n] } -> same_as<iter_reference_t<I>>;
        };
```

A type `I` supports the concept `random_access_iterator` if it supports the concept `bidirectional_iterator` and all the following requirements. For example, the requirement `{ i += n } -> same_as<I&>` as part of the `requires` expression means that for a value of type `I`, `{ i += n }` is a valid expression, and it returns a value of type `I&`. To complete the sorting story, `std::list` does support a `bidirectional_iterator`, and not a `random_access_iterator` that `std::sort` requires.

When you now use an algorithm that requires a `random_access_iterator`, but you only provide a `birectional_iterator`, you get a concise and readable error message saying that your iterator does not satisfy the concept `random_access_iterator`.

[4] https://en.cppreference.com/w/cpp/language/constraints

The Standard Template Library

The Essence of Generic Programming

I want to start this short historical detour with a quote from the invaluable book **From Mathematics to Generic Programming**[5], written by Alexander Stepanov (creator of the Standard Template Library) and Daniel Rose (information retrieval researcher): "*The essence of generic programming lies in the idea of concepts. A concept is a way of describing a family of related object types.*" These related object types can be integral types such as `bool`, `char`, or `int`. A concept embodies a set of requirements on related types such as their supported operations, semantics, and time and space complexity.

The Standard Template Library (STL) as a generic library is based on concepts. From a bird's-eye view, the STL consists of three components. Those are containers, algorithms that run on containers, and iterators that connect both of them.

Each container provides iterators that respect its structure, and the algorithms operate on these iterators. A container, such as a sequence container or an associative container, models a semi-open range. Access to the container's elements is provided through iterators, as well as iterating through them, and the equality comparison of them. The abstraction of the STL is based on concepts such as semi-open range and iterator and allow for transparent use of the containers and algorithms of the STL.

More generally, what are the advantages of concepts?

4.1.2 Advantages of Concepts

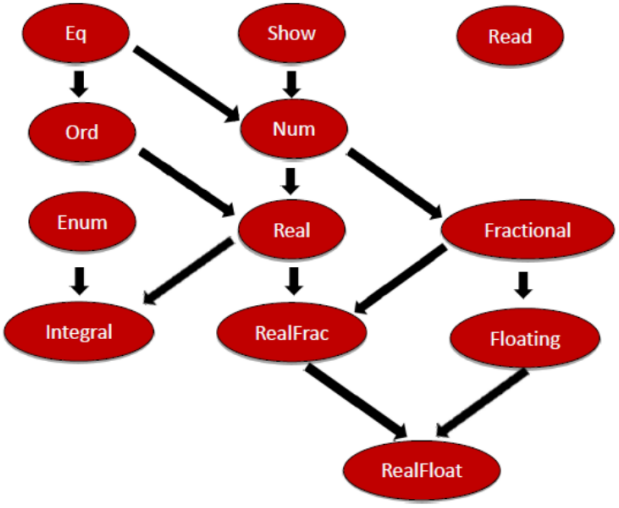
- Requirements for template parameters are part of the interface.

[5] https://www.fm2gp.com/

- The overloading of functions and specialization of class templates can be based on concepts.
- Concepts can be used for function templates, class templates, and generic member functions of classes or class templates.
- You get improved error messages because the compiler compares the requirements of the template parameters with the given template arguments.
- You can use predefined concepts or define your own.
- The usage of `auto` and concepts is unified. Instead of `auto`, you can use a concept.
- If a function declaration uses a concept, it automatically becomes a function template. Writing function templates is, therefore, as easy as writing a function.

4.1.3 The long, long History

The first time I heard about concepts was around 2005 - 2006. They reminded me of Haskell type classes. Type classes in Haskell are interfaces for similar types. Here is a part of Haskell's[6] type classes hierarchy.



Haskell Type Classes Hierarchy

But C++ concepts are different. Here are a few observations.

[6] https://en.wikipedia.org/wiki/Haskell_(programming_language)

- In Haskell, any type has to be an instance of a type class. In C++20, a type has to fulfill the requirements of a concept.
- Concepts can be used on non-type arguments of templates in C++. For example, numbers such as the value 5 are non-type arguments. For example, when you want to have a `std::array` of `ints` with 5 elements, you use the non-type argument 5: `std::array<int, 5> myArray`.
- Concepts add no run-time costs.

Originally, concepts were going to be the key feature of C++11, but they were removed during a standardization meeting in July 2009 in Frankfurt. The quote from Bjarne Stroustrup speaks for itself: "*The C++0x concept design evolved into a monster of complexity.*"[7]. A few years later, the next try was also not successful: concepts lite were removed from the C++17 standard. They finally become part of C++20.

4.1.4 Use of Concepts

Essentially, there are four ways to use a concept.

4.1.4.1 Four Ways to use a Concept

I apply the predefined concept `std::integral` in the program `conceptsIntegralVariations.cpp` in all four ways.

Four variations using the concept `std::integral`

```
1  // conceptsIntegralVariations.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template<typename T>
7  requires std::integral<T>
8  auto gcd(T a, T b) {
9      if( b == 0 ) return a;
10     else return gcd(b, a % b);
```

[7] https://isocpp.org/blog/2013/02/concepts-lite-constraining-templates-with-predicates-andrew-sutton-bjarne-s

```
11  }
12
13  template<typename T>
14  auto gcd1(T a, T b) requires std::integral<T> {
15      if( b == 0 ) return a;
16      else return gcd1(b, a % b);
17  }
18
19  template<std::integral T>
20  auto gcd2(T a, T b) {
21      if( b == 0 ) return a;
22      else return gcd2(b, a % b);
23  }
24
25  auto gcd3(std::integral auto a, std::integral auto b) {
26      if( b == 0 ) return a;
27      else return gcd3(b, a % b);
28  }
29
30  int main(){
31
32      std::cout << '\n';
33
34      std::cout << "gcd(100, 10)= "  <<  gcd(100, 10)  << '\n';
35      std::cout << "gcd1(100, 10)= " <<  gcd1(100, 10)  << '\n';
36      std::cout << "gcd2(100, 10)= " <<  gcd2(100, 10)  << '\n';
37      std::cout << "gcd3(100, 10)= " <<  gcd3(100, 10)  << '\n';
38
39      std::cout << '\n';
40
41  }
```

Thanks to the header `<concepts>` in line 3, I can use the concept `std::integral`. The concept is fulfilled if `T` is the type integral[8]. The function name `gcd` stands for the

[8] https://en.cppreference.com/w/cpp/types/is_integral

greatest-common-divisor algorithm based on the Euclidean[9] algorithm.

Here are the four ways to use concepts:

- Requires clause (line 6)
- Trailing requires clause (line 13)
- Constrained template parameter (line 19)
- Abbreviated function template (line 25)

For simplicity reasons, each function template returns just `auto`. There is a semantic difference between the function templates `gcd`, `gcd1`, `gcd2`, and the function `gcd3`. In the case of `gcd`, `gcd1`, or `gcd2`, the arguments `a` and `b` must have the same type. This does not hold for the function `gcd3`. Parameters `a` and `b` can have different types, but must both fulfil the concept `integral`.

```
gcd(100, 10)= 10
gcd1(100, 10)= 10
gcd2(100, 10)= 10
gcd3(100, 10)= 10
```

Use of the concept `std::integral`

The functions `gcd` and `gcd1` use requires clauses. Requires clauses are more powerful than you may think. Let me discuss more details to requires clauses.

4.1.4.2 Requires Clause

The previous program, `conceptsIntegralVariations.cpp`, exemplifies that you can use a concept to define a function or function template. Of course, there are more use cases. For completeness, I want to add that you can specify the return type of a function or a function template using concepts.

The keyword `requires` introduces a requires clause which specifies constraints on a template argument (`gcd`) or on a function declaration (`gcd1`). `requires` must be followed by a compile-time predicate such as a named concept (`gcd`), a conjunction/disjunction of named concepts, or a requires expression.

The compile-time predicate can also be an expression:

[9] https://en.wikipedia.org/wiki/Euclid

Using a compile-time predicate in a requires clause

```
1  // requiresClause.cpp
2
3  #include <iostream>
4
5  template <unsigned int i>
6  requires (i <= 20)
7  int sum(int j) {
8      return i + j;
9  }
10
11
12 int main() {
13
14     std::cout << '\n';
15
16     std::cout << "sum<20>(2000): " << sum<20>(2000) << '\n',
17     // std::cout << "sum<23>(2000): " << sum<23>(2000) << '\n',  // ERR\
18 OR
19
20     std::cout << '\n';
21
22 }
```

The compile-time predicate used in line 6 exemplifies an interesting point: the requirement is applied on the non-type `i`, and not on a type as usual.

```
sum<20>(2000): 2020
```

Compile-time predicates in a requires clause

When you use line 17, the clang compiler reports the following error:

```
<source>:17:39: error: no matching function for call to 'sum'
    std::cout << "sum<23>(2000): " << sum<23>(2000) << '\n',  // ERROR
                                      ^~~~~~~
<source>:7:5: note: candidate template ignored: constraints not satisfied [with i = 23]
int sum(int j) {
    ^
<source>:6:11: note: because '23U <= 20' (23 <= 20) evaluated to false
requires (i <= 20)
          ^
```

Failing compile time predicates in a requires clauses

Avoid Compile-Time Predicates in Requires Clauses

When you constrain template parameter or function templates using concepts, you should use named concepts or combinations of them. Concepts are meant to be semantic categories, but not syntactic constraints like `i <= 20`. Giving concepts a name enables their reuse.

4.1.4.3 Concepts as Return Type of a Function

Here are the definitions of the function template `gcd` and the function `gcd1` using concepts as return types.

Using a concept as return type

```
template<typename T>
requires std::integral<T>
std::integral auto gcd(T a, T b) {
    if( b == 0 ) return a;
    else return gcd(b, a % b);
}

std::integral auto gcd1(std::integral auto a, std::integral auto b) {
    if( b == 0 )return a;
    else return gcd1(b, a % b);
}
```

4.1.4.4 Use-Cases for Concepts

First and foremost, concepts are compile-time predicates. A compile-time predicate is a function that is executed at compile time and returns a boolean. Before I dive into the various use cases of concepts, I want to demystify concepts and present them simply as functions returning a boolean at compile time.

4.1.4.4.1 Compile-Time Predicates

A concept can be used in a control structure, which is executed at run time or compile time.

Concepts as compile-time predicates

```
// compileTimePredicate.cpp

#include <compare>
#include <iostream>
#include <string>
#include <vector>

struct Test{};

int main() {

    std::cout << '\n';

    std::cout << std::boolalpha;

    std::cout << "std::three_way_comparable<int>: "
              << std::three_way_comparable<int> << "\n";

    std::cout << "std::three_way_comparable<double>: ";
    if (std::three_way_comparable<double>) std::cout << "True";
    else std::cout << "False";

    std::cout << "\n\n";
```

```
24
25      static_assert(std::three_way_comparable<std::string>);
26
27      std::cout << "std::three_way_comparable<Test>: ";
28      if constexpr(std::three_way_comparable<Test>) std::cout << "True";
29      else std::cout << "False";
30
31      std::cout << '\n';
32
33      std::cout << "std::three_way_comparable<std::vector<int>>: ";
34      if constexpr(std::three_way_comparable<std::vector<int>>) std::cout\
35   << "True";
36      else std::cout << "False";
37
38      std::cout << '\n';
39
40  }
```

In the program above, I use the concept `std::three_way_comparable<T>`, which checks at compile time if `T` supports the six comparison operators. Being a compile-time predicate means, that `std::three_way_comparable` can be used at run time (lines 16 and 20) or at compile time. `static_assert` (line 25) and `constepr if`[10] (lines 28 and 34) are evaluated at compile time.

```
std::three_way_comparable<int>: true
std::three_way_comparable<double>: True

std::three_way_comparable<Test>: False
std::three_way_comparable<std::vector<int>>: True
```

Concepts as compile-time predicates

After this short detour on concepts as compile-time predicates, let me continue this section with the various use cases of concepts. The concepts' applications are not too

[10] https://en.cppreference.com/w/cpp/language/if

elaborate, and I mainly use predefined concepts, which I describe in more depth in the section predefined concepts.

4.1.4.4.2 Class Templates

The class template `MyVector` requires that its template parameter `T` be `regular`, meaning that `T` behaves such as an `int`. The formal definition of `regular` is provided in the define concepts section.

Using a concept in a class definition

```
1  // conceptsClassTemplate.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template <std::regular T>
7  class MyVector{};
8
9  int main() {
10
11     MyVector<int> myVec1;
12     MyVector<int&> myVec2;  // ERROR because a reference is not regular
13
14 }
```

Line 12 causes a compile-time error because a reference is not `regular`. Here is the essential part of the GCC compiler message:

```
<source>:13:18: error: template constraint failure for 'template<class T>  requires  regular<T> class MyVector'
   13 |     MyVector<int&> myVec2;
```

A reference is not `regular`

4.1.4.4.3 Generic Member Functions

In this example, I add a generic `push_back` member function to the class `MyVector`. The `push_back` requires that its arguments be copyable.

Using a concept in a generic member function

```
1  // conceptMemberFunction.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  struct NotCopyable {
7      NotCopyable() = default;
8      NotCopyable(const NotCopyable&) = delete;
9  };
10
11 template <typename T>
12 struct MyVector{
13     void push_back(const T&) requires std::copyable<T> {}
14 };
15
16 int main() {
17
18     MyVector<int> myVec1;
19     myVec1.push_back(2020);
20
21     MyVector<NotCopyable> myVec2;
22     myVec2.push_back(NotCopyable());  // ERROR because not copyable
23
24 }
```

The compilation fails intentionally in line 22. Instances of `NotCopyable` are not copyable because the copy constructor is declared as deleted.

4.1.4.4.4 Variadic Templates

You can use concepts in variadic templates.

Applying concepts to variadic templates

```
1  // allAnyNone.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template<std::integral... Args>
7  bool all(Args... args) { return (... && args); }
8
9  template<std::integral... Args>
10 bool any(Args... args) { return (... || args); }
11
12 template<std::integral... Args>
13 bool none(Args... args) { return not(... || args); }
14
15 int main(){
16
17     std::cout << std::boolalpha << '\n';
18
19     std::cout << "all(5, true, false): " << all(5, true, false) << '\n';
20
21     std::cout << "any(5, true, false): " << any(5, true, false) << '\n';
22
23     std::cout << "none(5, true, false): " << none(5, true, false) << '\\
24 n';
25
26 }
```

The definitions of the function templates above are based on fold expressions. C++11 supports variadic templates that can accept an arbitrary number of template arguments. The arbitrary number of template parameters is held by a so-called parameter pack. Additionally, with C++17 you can directly reduce a parameter pack with a binary operator. This reduction is called a fold expression[11]. In this example, the logical and `&&` (line 7), the logical or `||` (line 10), and the negation of the logical

[11] https://www.modernescpp.com/index.php/fold-expressions

or (line 13) are applied as binary operators. Furthermore, `all`, `any`, and `none` requires from their type parameters that they have to support the concept `std::integral`.

```
all(5, true, false): false
any(5, true, false): true
none(5, true, false): false
```

Applying concepts onto a fold expression

4.1.4.4.5 Overloading

`std::advance`[12] is an algorithm of the Standard Template Library. It increments a given iterator `iter` by `n` elements. Based on the capabilities of the given iterator, a different advance strategy could be used. For example, a `std::forward_list` supports an iterator that can only advance in one direction, while a `std::list` supports a bidirectional iterator, and a `std::vector` supports a random access iterator. Consequently, for an iterator provided by a `std::forward_list` or `std::list`, a call to `std::advance(iter, n)` has to be incremented `n` times (see the structure of a `std::list`). This time complexity does not hold for a `std::random_access_iterator` provided by a `std::vector`. The number `n` can just be added to the iterator. A linear time complexity `O(n)` becomes, therefore, a constant complexity `O(1)`. To distinguish iterator types, concepts can be used. The program `conceptsOverloadingFunctionTemplates.cpp` should give you the general idea.

Overloading function templates on concepts

```
1  // conceptsOverloadingFunctionTemplates.cpp
2
3  #include <concepts>
4  #include <iostream>
5  #include <forward_list>
6  #include <list>
7  #include <vector>
8
9  template<std::forward_iterator I>
10 void advance(I& iter, int n){
```

[12] https://en.cppreference.com/w/cpp/iterator/advance

```
11      std::cout << "forward_iterator" << '\n';
12  }
13
14  template<std::bidirectional_iterator I>
15  void advance(I& iter, int n){
16      std::cout << "bidirectional_iterator" << '\n';
17  }
18
19  template<std::random_access_iterator I>
20  void advance(I& iter, int n){
21      std::cout << "random_access_iterator" << '\n';
22  }
23
24  int main() {
25
26      std::cout << '\n';
27
28      std::forward_list forwList{1, 2, 3};
29      std::forward_list<int>::iterator itFor = forwList.begin();
30      advance(itFor, 2);
31
32      std::list li{1, 2, 3};
33      std::list<int>::iterator itBi = li.begin();
34      advance(itBi, 2);
35
36      std::vector vec{1, 2, 3};
37      std::vector<int>::iterator itRa = vec.begin();
38      advance(itRa, 2);
39
40      std::cout << '\n';
41  }
```

The three variations of the function `advance` are overloaded on the concepts `std::forward_iterator` (line 9), `std::bidirectional_iterator` (line 14), and `std::random_access_iterator` (line 19). The compiler chooses the best-fitting overload. This means that for

a `std::forward_list` (line 28) the overload based on the concept `std::forward_list`, for a `std::list` (line 32) the overload based on the concept `std::bidirectional_iterator`, and for a `std::vector` (line 36) the overload based on the concept `std::random_access_iterator` is used.

```
forward_iterator
bidirectional_iterator
random_access_iterator
```

Overloading function templates on concepts

A `std::random_access_iterator` is a `std::bidirectional_iterator`, and `std::bidirectional_iterator` is a `std::forward_iterator`.

4.1.4.4.6 Template Specialization

You can also specialize templates using concepts.

Template specialization on concepts

```
1  // conceptsSpecialization.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template <typename T>
7  struct Vector {
8      Vector() {
9          std::cout << "Vector<T>" << '\n';
10     }
11 };
12
13 template <std::regular Reg>
14 struct Vector<Reg> {
15     Vector() {
16         std::cout << "Vector<std::regular>" << '\n';
17     }
18 };
```

```
19
20 int main() {
21
22     std::cout << '\n';
23
24     Vector<int> myVec1;
25     Vector<int&> myVec2;
26
27     std::cout << '\n';
28
29 }
```

When instantiating the class template, the compiler chooses the most specialized one. This means for the call `Vector<int> myVec` (line 24), the partial template specialization for `std::regular` (line 13) is chosen. A reference `Vector<int&> myVec2` (line 25) is not `regular`. Consequently, the primary template (line 6) is chosen.

```
Vector<std::regular>
Vector<T>
```

Partial template specialization of concepts

4.1.4.4.7 Using More than One Concept

So far, the uses of the concepts were straightforward, but most of the time more than one concept is used at the same time.

Using more than one concept

```
template<typename Iter, typename Val>
    requires std::input_iterator<Iter>
          && std::equality_comparable<Value_type<Iter>, Val>
Iter find(Iter b, Iter e, Val v)
```

`find` requires for the iterator `Iter` and its comparison with `Val` that

- the Iterator has to be an input iterator;
- the Iterator's value type must be equality comparable with `Val`.

The same restriction on the iterator can also be expressed as a constrained template parameter.

Using more than one concept

```
template<std::input_iterator Iter, typename Val>
    requires std::equality_comparable<Value_type<Iter>, Val>
Iter find(Iter b, Iter e, Val v)
```

4.1.5 Constrained and Unconstrained Placeholders

First, let me tell you about an asymmetry in C++14.

4.1.5.1 The Big Asymmetry in C++14

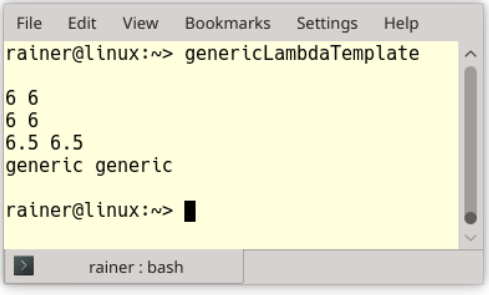
I often have a discussion in my classes, that goes the following way. With C++14, we had generic lambdas. Generic lambdas are lambdas that use `auto` instead of a concrete type.

Comparison of a generic lambda and a function template

```
1  // genericLambdaTemplate.cpp
2
3  #include <iostream>
4  #include <string>
5
6  auto addLambda = [](auto fir, auto sec){ return fir + sec; };
7
8  template <typename T, typename T2>
9  auto addTemplate(T fir, T2 sec){ return fir + sec; }
10
11 int main(){
12
```

```
13      std::cout << std::boolalpha << '\n';
14
15      std::cout << addLambda(1, 5) << " " << addTemplate(1, 5) << '\n';
16      std::cout << addLambda(true, 5) << " " << addTemplate(true, 5) << '\
17  \n';
18      std::cout << addLambda(1, 5.5) << " " << addTemplate(1, 5.5) << '\n\
19  ';
20
21      const std::string fir{"ge"};
22      const std::string sec{"neric"};
23      std::cout << addLambda(fir, sec) << " " << addTemplate(fir, sec) <<\
24   '\n';
25
26      std::cout << '\n';
27
28  }
```

The generic lambda (line 6) and the function template (line 8) produce the same results.



Use of a generic lambda and a function template

Generic lambdas introduce a new way to define function templates. In my classes, I'm often asked: Can we use `auto` in functions to get function templates? Not with C++14, but you can with C++20. In C++20, you can use unconstrained placeholders (`auto`) or constrained placeholders (concepts) in function declarations to automatically get function templates. The rule for applying is as simple as it could be. In each place

where you can use an unconstrained placeholder `auto`, you can use a concept. I will detail this fully in the section on abbreviated function templates.

4.1.5.2 Placeholders

Use of constrained placeholders instead of unconstrained placeholders

```
// placeholders.cpp

#include <concepts>
#include <iostream>
#include <vector>

std::integral auto getIntegral(int val){
    return val;
}

int main(){

    std::cout << std::boolalpha << '\n';

    std::vector<int> vec{1, 2, 3, 4, 5};
    for (std::integral auto i: vec) std::cout << i << " ";
    std::cout << '\n';

    std::integral auto b = true;
    std::cout << b << '\n';

    std::integral auto integ = getIntegral(10);
    std::cout << integ << '\n';

    auto integ1 = getIntegral(10);
    std::cout << integ1 << '\n';

    std::cout << '\n';

}
```

The concept `std::integral` can be used as a return type (line 7), in a range-based for loop (line 16), or as a type for variable `b` (line 19), or variable `integ` (line 22). To see the symmetry between `auto` and concepts, line 25 uses `auto` alone instead of `std::integral auto`, which is used on line 22. Hence, `integ1` can accept a value of any type.

```
1 2 3 4 5
true
10
10
```

Constrained placeholders instead of unconstrained placeholders in action

4.1.6 Abbreviated Function Templates

With C++20, you can use an unconstrained placeholder (`auto`) or a constrained placeholder (concept) in a function declaration and this function declaration automatically becomes a function template.

Abbreviated function templates

```
1  // abbreviatedFunctionTemplates.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template<typename T>
7  requires std::integral<T>
8  T gcd(T a, T b) {
9      if( b == 0 ) return a;
10     else return gcd(b, a % b);
11 }
12
13 template<typename T>
14 T gcd1(T a, T b) requires std::integral<T> {
15     if( b == 0 ) return a;
```

```
16      else return gcd1(b, a % b);
17  }
18
19  template<std::integral T>
20  T gcd2(T a, T b) {
21      if( b == 0 ) return a;
22      else return gcd2(b, a % b);
23  }
24
25  std::integral auto gcd3(std::integral auto a, std::integral auto b) {
26      if( b == 0 ) return a;
27      else return gcd3(b, a % b);
28  }
29
30  auto gcd4(auto a, auto b){
31      if( b == 0 ) return a;
32      return gcd4(b, a % b);
33  }
34
35  int main() {
36
37      std::cout << '\n';
38
39      std::cout << "gcd(100, 10)= "  <<  gcd(100, 10)  << '\n';
40      std::cout << "gcd1(100, 10)= " <<  gcd1(100, 10)  << '\n';
41      std::cout << "gcd2(100, 10)= " <<  gcd2(100, 10)  << '\n';
42      std::cout << "gcd3(100, 10)= " <<  gcd3(100, 10)  << '\n';
43      std::cout << "gcd4(100, 10)= " <<  gcd4(100, 10)  << '\n';
44
45      std::cout << '\n';
46
47  }
```

The definitions of the function templates `gcd` (line 6), `gcd1` (line 13), and `gcd2` (line 19) are the ones I already presented in section Four ways to use a concept.

gcd uses a requires clause, gcd1 a trailing requires clause and gcd2 a constrained template parameter. Now to something new. Function template gcd3 has the concept std::integral as a type parameter and becomes, therefore, a function template with restricted type parameters. In contrast, gcd4 is equivalent to function templates with no restriction on its type parameters. The syntax used in gcd3 and gcd4 to create a function template is called abbreviated function templates syntax.

```
gcd(100, 10)=  10
gcd1(100, 10)= 10
gcd2(100, 10)= 10
gcd3(100, 10)= 10
gcd4(100, 10)= 10
```

Constrained

Let me stress this symmetry by demonstrying it in another example below.

By using auto as a type parameter, the function add becomes a function template and is equivalent to the equally-named function template add.

The equivalent function and function template add

```
template<typename T, typename T2>
auto add(T fir, T2 sec) {
    return fir + sec;
}

auto add(auto fir, auto sec) {
    return fir + sec;
}
```

Accordingly, due to the usage of the concept std::integral, the function sub is equivalent to the function template sub.

The equivalent function and function template sub

```
template<std::integral T, std::integral T2>
std::integral auto sub(T fir, T2 sec) {
    return fir - sec;
}

std::integral auto sub(std::integral auto fir, std::integral auto sec) \
{
    return fir - sec;
}
```

The function and the function template can have arbitrary types. This means both types can be different but must be integral. For example, a call `sub(100, 10)` and also `sub(100, true)` would be valid.

There is one interesting feature still missing in the abbreviated function templates syntax: you can overload on `auto` or concepts.

4.1.6.1 Overloading

The following functions `overload` are overloaded on `auto`, on the concept `std::integral`, and on the type `long`.

Abbreviated function templates and overloading

```
1  // conceptsOverloading.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  void overload(auto t){
7      std::cout << "auto : " << t << '\n';
8  }
9
10 void overload(std::integral auto t){
11     std::cout << "Integral : " << t << '\n';
```

```
12  }
13
14  void overload(long t){
15      std::cout << "long : " << t << '\n';
16  }
17
18  int main(){
19
20      std::cout << '\n';
21
22      overload(3.14);
23      overload(2010);
24      overload(2020L);
25
26      std::cout << '\n';
27
28  }
```

The compiler chooses the overload on `auto` (line 6) with a double, the overload on the concept `std::integral` (line 10) with an `int`, and the overload on `long` (line 14) with a `long`.

```
auto : 3.14
Integral : 2010
long : 2020
```

Abbreviated function templates and overloading

What we don't get: Template Introduction

Maybe you are missing one feature in this chapter on concepts: template introduction. Template introduction was part of the technical specification on concepts, TS ISO/IEC TS 19217:2015[13], and was an experimental implementation of concepts. GCC 6[14] fully implemented the concepts TS. Besides the syntactic differences from concepts in C++20, the concepts TS supported a concise way of defining templates.

In the following example assume that `Integral` is a concept.

Template introduction in the concepts TS

```
Integral{T}
Integral gcd(T a, T b){
   if( b == 0 ){ return a; }
   else{
       return gcd(b, a % b);
   }
}

Integral{T}
class ConstrainedClass{};
```

This small code snippet above used template introduction in two ways. First, to define a function template with a constrained template parameter; second, to define a class template with a constrained template parameter. Template introduction had one limitation. You could only use it with a constrained template parameter (concept), but not with an unconstrained template parameter (`auto`). This asymmetry could easily be overcome by defining a concept that always returns `true`:

The concept `Generic` is always fulfilled

```
template<typename T>
concept bool Generic(){
   return true;
}
```

Don't be irritated, I used in the example the concepts TS syntax to define the `Generic` concept. The C++20 syntax is slightly more concise. Read more details of the C++20 syntax in section Defining Concepts.

[13] https://www.iso.org/standard/64031.html

4.1.7 Predefined Concepts

The golden rule "Don't reinvent the wheel" also applies to concepts. The C++ Core Guidelines[15] are very clear about this rule: T.11: Whenever possible, use standard concepts. Consequently, I want to give you an overview of the important predefined concepts. I intentionally ignore any special or auxiliary concepts.

All predefined concepts are detailed in the latest C++20 working draft, N4860[16], and finding them all can be quite a challenge! Most of the concepts are in chapter 18 (concepts library) and chapter 24 (ranges library). Additionally, a few concepts are in chapter 17 (language support library), chapter 20 (general utilities library), chapter 23 (iterators library), and chapter 26 (numerics library). The C++20 draft N4860 also has an index to all library concepts and shows how the concepts are implemented.

4.1.7.1 Language Support Library

This section discusses an interesting concept, `three_way_comparable`. It is used to support the three-way comparison operator. It is specified in the header `<compare>`.

More formally, let `a` and `b` be values of type `T`. This values are `three_way_comparable` only if:

- `(a <=> b == 0) == bool(a == b)` is `true`
- `(a <=> b != 0) == bool(a != b)` is `true`
- `((a <=> b) <=> 0)` and `(0 <=> (b <=> a))` are equal
- `(a <=> b < 0) == bool(a < b)` is `true`
- `(a <=> b > 0) == bool(a > b)` is `true`
- `(a <=> b <= 0) == bool(a <= b)` is `true`
- `(a <=> b >= 0) == bool(a >= b)` is `true`

4.1.7.2 Concepts Library

The most frequently used concepts can be found in the concepts library. They are defined in the `<concepts>` header.

[14] https://en.wikipedia.org/wiki/GNU_Compiler_Collection
[15] https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines
[16] https://isocpp.org/files/papers/N4860.pdf

4.1.7.2.1 Language-related concepts

This section has about 15 concepts that should be self-explanatory. These concepts express relationships between types, type classifications, and fundamental type properties. Their implementation is often directly based on the corresponding function from the type-traits library[17]. Where deemed necessary, I provide additional explanation.

- `same_as`
- `derived_from`
- `convertible_to`
- `common_reference_with`: `common_reference_with<T, U>` must be well-formed and `T` and `U` must be convertible to a reference type `C`, where `C` is the same as `common_reference_t<T, U>`
- `common_with`: similar to `common_reference_with`, but the common type `C` is the same as `common_type_t<T, U>` and may not be a reference type
- `assignable_from`
- `swappable`

4.1.7.2.2 Arithmetic Concepts

- `integral`
- `signed_integral`
- `unsigned_integral`
- `floating_point`

The standard's definition of the arithmetic concepts is straightforward:

[17] https://en.cppreference.com/w/cpp/header/type_traits

```
template<class T>
concept integral = is_integral_v<T>;

template<class T>
concept signed_integral = integral<T> && is_signed_v<T>;

template<class T>
concept unsigned_integral = integral<T> && !signed_integral<T>;

template<class T>
concept floating_point = is_floating_point_v<T>;
```

4.1.7.2.3 Lifetime Concepts

- `destructible`
- `constructible_from`
- `default_constructible`
- `move_constructible`
- `copy_constructible`

4.1.7.2.4 Comparison Concepts

- `equality_comparable`
- `totally_ordered`

Maybe you know it from your mathematics studies: For values `a`, `b`, and `c` of type `T`, `T` models `totally_ordered` if and only if

- Exactly one of `bool(a < b)`, `bool(a > b)`, or `bool(a == b)` is `true`
- If `bool(a < b)` and `bool(b < c)`, then `bool(a < c)`
- `bool(a > b) == bool(b < a)`
- `bool(a <= b) == !bool(b < a)`
- `bool(a >= b) == !bool(a < b)`

4.1.7.2.5 Object Concepts

- movable
- copyable
- semiregular
- regular

Here are the concise definitions of the four concepts:

```
template<class T>
concept movable = is_object_v<T> && move_constructible<T> &&
                  assignable_from<T&, T> && swappable<T>;

template<class T>
concept copyable = copy_constructible<T> && movable<T> &&
                   assignable_from<T&, T&> &&
                   assignable_from<T&, const T&> && assignable_from<T&,\
 const T>;

template<class T>
concept semiregular = copyable<T> && default_initializable<T>;

template<class T>
concept regular = semiregular<T> && equality_comparable<T>;
```

I have to add a few words. The concept `movable` requires for `T` that `is_object_v<T>` holds. From the definition of the type-trait `is_object<T>` this means that `T` is either a scalar, an array, a union, or a class.

I implement the concept `semiregular` and `regular` in the section define concepts. Informally, a `semiregular` type behaves similar to an `int`, and a `regular` type behaves similarly to an `int` and can be compared using ==.

4.1.7.2.6 Callable Concepts

- invocable

- `regular_invocable`: a type models `invocable` and equality-preserving, and does not modify the function arguments; equality-preserving means the it produces the same output when given the same input
- `predicate`: a type models a predicate if it models `invocable` and returns a boolean

4.1.7.3 General Utilities Library

This chapter in the standard has only special memory concepts; therefore I don't refer to them here.

4.1.7.4 Iterators Library

The iterators library has many important concepts. They are defined in the `<iterator>` header. Here are the iterator categories:

- `input_iterator`
- `output_iterator`
- `forward_iterator`
- `bidirectional_iterator`
- `random_access_iterator`
- `contiguous_iterator`

The six categories of iterators correspond to the respective iterator concepts. The table below provides two interesting pieces of information. For the three most prominent iterator categories, the table shows their properties and the associated standard library containers.

Properties and Containers of each iterator category

Iterator Category	Properties	Containers

`std::forward_iterator`	`++It`, `It++` , `*It`
	`It == It2`, `It != It2`
		`std::unordered_set`
		`std::unordered_map`
		`std::unordered_multiset`
		`std::unordered_multimap`
		`std::forward_list`

`std::bidirectional_iterator`	`--It`, `It--`	`std::set`
		`std::map`
		`std::multiset`
		`std::multimap`
		`std::list`

`std::random_access_iterator`	`It[i]`
	`It += n`, `It -= n`
	`It + n` , `It - n`
	`n + It`
	`It - It2`
	`It < It2`, `It <= It2`
	`It > It2`, `It >= It2`
		`std::array`
		`std::vector`
		`std::deque`
		`std::string`

The following relation holds: A random-access-iterator is a bidirectional iterator, and a bidirectional iterator is a forward iterator. A contiguous iterator is a random-access-iterator and requires that the elements of the container are stored contiguously in memory. This means `std::array`, `std::vector`, and `std::string`, but not `std::deque`, support contiguous iterators.

4.1.7.4.1 Algorithm Concepts

- `permutable`: in-place reordering of elements is possible
- `mergeable`: merging sorted sequences into an output sequence is possible
- `sortable`: permuting a sequence into an ordered sequence is possible

4.1.7.5 Ranges Library

The ranges library contains the concepts critical to the ranges and views features. They are similar to the concepts in the iterators library and are defined in the `<ranges>` header.

4.1.7.5.1 Ranges

- `range`: A range specifies a group of items that you can iterate over. It provides a begin iterator and an end sentinel. Of course, the containers of the STL are ranges.

There are further refinements for `std::ranges::range`.

- `input_range`: specifies a range whose iterator type satisfies `input_iterator` (e.g. can iterate from beginning to end at least once)
- `output_range`: specifies a range whose iterator type satisfies `output_iterator`
- `forward_range`: specifies a range whose iterator type satisfies `forward_iterator` (can iterate from beginning to end more than once)
- `bidirectional_range` : specifies a range whose iterator type satisfies `bidirectional_iterator` (can iterate forward and backward more than once)
- `random_access_range`: specifies a range whose iterator type satisfies `random_access_iterator` (can jump in constant time to an arbitrary element with the index operator [])
- `contiguous_range`: specifies a range whose iterator type satisfies `contiguous_iterator` (elements are stored consecutively in memory)

Each container of the Standard Template Library supports a specific range. The supported range specifies the capabilities of its iterators.

Properties and containers of each range concept

Concept	Properties	Containers

`std::ranges::input_range`	`++It`, `It++` , `*It`
	`It == It2`, `It != It2`
		`std::unordered_set`
		`std::unordered_map`
		`std::unordered_multiset`
		`std::unordered_multmap`
		`std::forward_list`

`std::ranges::bidirectional_range`	`--It`, `It--`	`std::set`
		
		`std::map`
		`std::multiset`
		`std::multimap`
		`std::list`

`std::ranges::random_access_range`	`It[i]`
	
	`It += n`, `It -= n`
	`It + n` , `It - n`
	`n + It`
	`It - It2`
	`It < It2`, `It <= It2`
	`It > It2`, `It >= It2`
		`std::deque`

`std::ranges::contiguous_range`	`It[i]`
	`It += n`, `It -= n`
	`It + n` , `It - n`
	`n + It`
	`It - It2`
	`It < It2`, `It <= It2`
	`It > It2`, `It >= It2`
		`std::array`
		`std::vector`
		`std::string`

A container supporting the `std::ranges::contiguous_range` concept, supports all revious mentioned concepts in the table such as `std::ranges::random_access_range`, `std::ranges::bidirectional_range`, and `std::ranges::input_range`. The same holds for all other ranges.

4.1.7.5.2 Views

A `std::ranges::view` typically something that you apply on a range and performs some operation. A view does not own data and the time a view takes to copy, move, or assign is constant. Here is a quote from Eric Niebler's range-v3 implementation, which is the basis for the C++20 ranges: "*Views are composable adaptations of ranges where the adaptation happens lazily as the view is iterated.*"

4.1.7.6 Numeric Library

The numeric library provides the concept of a `uniform_random_bit_generator` that is defined in the header `<random>`. A `uniform_random_bit_generator g` of type `G` must return uniformly-distributed unsigned integers. Additionally, a uniform random-bit generator `g` of type `G` has to support the member functions `G::min` and `G::max`.

4.1.8 Defining Concepts

When the concept you are looking for is not one of the predefined concepts in C++20, you must define your own concept. In this section I will define a few concepts which will be distinguishable from the predefined concepts through the use of CamelCase syntax. Consequently, my concept for a signed integral is named `SignedIntegral`, whereas the C++ standard concept goes by the name `signed_integral`.

The syntax for defining a concept is straightforward:

Concept definition

```
template <template-parameter-list>
concept concept-name = constraint-expression;
```

A concept definition starts with the keyword `template` and has a template parameter list. The second line is more interesting. It uses the keyword `concept` followed by the concept name and the constraint expression.

A `constraint-expression` can either be:

- A logical combination of other concepts or compile-time predicates

  - Logical combination can be built out of conjunctions (`&&`), disjunctions (`||`), or negations (`!`)
  - Compile-time predicates are callables that return a boolean value at compile time
- A requires expression
  - Simple requirements
  - Type requirements
  - Compound requirements
  - Nested requirements

In the next two sections I will demonstrate various ways of defining concepts.

4.1.8.1 A Logical Combination of other Concepts and Compile-Time Predicates

You can combine concepts and compile time predicates using conjunctions (`&&`) and disjunctions (`||`). When building your logical combination, you can negate components by using the exclamation mark (`!`). Thanks to the many compile-time predicates of the type-traits library[18], you have at your disposal all tools required to build powerful concepts.

[18] https://en.cppreference.com/w/cpp/header/type_traits

Don't define Concepts Recursively or try to Constrain them

A recursive definition of a concept is not valid:

Recursively defining a concept

```
template<typename T>
concept Recursive = Recursive<T*>;
```

The GCC compiler complains in this case that `'Recursive' was not declared in this scope`.

When you try to constrain a concept such as in the following code snippet, the GCC compiler unambiguously complains that `a concept cannot be constrained`.

Constraining a concept

```
template<typename T>
concept AlwaysTrue = true;

template<typename T>
requires AlwaysTrue<T>
concept Error = true;
```

Let's start with the concepts `Integral`, `SignedIntegral`, and `UnsignedIntegral`.

The concepts `Integral`, `SignedIntegral`, and `UnsignedIntegral`

```
1 template <typename T>
2 concept Integral = std::is_integral<T>::value;
3
4 template <typename T>
5 concept SignedIntegral = Integral<T> && std::is_signed<T>::value;
6
7 template <typename T>
8 concept UnsignedIntegral = Integral<T> && !SignedIntegral<T>;
```

I used the type-traits function `std::is_integral`[19] to define the concept `Integral` (line 2). Thanks to the function `std::is_signed`, I refine the concepts `Integral` to the concept `SignedIntegral` (line 4). Finally, negating the concept `SignedIntegral` gives me the concept `UnsignedIntegral` (line 7).

Okay, let's try it out.

Use of the concepts `Integral`, `SignedIntegral`, and `UnsignedIntegral`

```
1  // SignedUnsignedIntegrals.cpp
2
3  #include <iostream>
4  #include <type_traits>
5
6  template <typename T>
7  concept Integral = std::is_integral<T>::value;
8
9  template <typename T>
10 concept SignedIntegral = Integral<T> && std::is_signed<T>::value;
11
12 template <typename T>
13 concept UnsignedIntegral = Integral<T> && !SignedIntegral<T>;
14
15 void func(SignedIntegral auto integ) {
16     std::cout << "SignedIntegral: " << integ << '\n';
17 }
18
19 void func(UnsignedIntegral auto integ) {
20     std::cout << "UnsignedIntegral: " << integ << '\n';
21 }
22
23 int main() {
24
25     std::cout << '\n';
26
27     func(-5);
```

[19] https://en.cppreference.com/w/cpp/types/is_integral

```
28      func(5u);
29
30      std::cout << '\n';
31
32  }
```

I used the abbreviated function-template syntax to overload the function `func` on the concept `SignedIntegral` (line 15) and `UnsignedIntegral` (line 19). The compiler chooses the expected overload:

```
SignedIntegral: -5
UnsignedIntegral: 5
```

Use of the concepts `SignedIntegral`, and `UnsignedIntegral`

For completeness reasons, the following concept `Arithmetic` uses disjunction.

The concept `Arithmetic`

```
template <typename T>
concept Arithmetic = std::is_integral<T>::value || std::is_floating_poi\
nt<T>::value;
```

4.1.8.2 Requires Expressions

Thanks to `requires` expressions, you can define powerful concepts. A `requires` expression has the following form:

Requires expression

```
requires (parameter-list(optional)) {requirement-seq}
```

- `parameter-list`: A comma-separated list of parameters, such as in a function declaration
- `requirement-seq`: A sequence of requirements, consisting of simple, type, compound, or nested requirements

4.1.8.2.1 Simple Requirements

The following concept `Addable` is a simple requirement:

The concept `Addable`

```
template<typename T>
concept Addable = requires (T a, T b) {
    a + b;
};
```

The concept `Addable` requires that the addition `a + b` of two values of the same type `T` is possible.

Avoid Anonymous Concepts: `requires requires`

You can define an anonymous concept and directly use it. Avoid it. This makes your code hard to read and you cannot reuse your concepts.

An anonymous concept for adding two concepts

```
template<typename T>
    requires requires (T x) { x + x; }
T add1(T a, T b) { return a + b; }
```

The function template defines its concept ad-hoc. `add1` uses a requires expression inside a requires clause. The anonymous concept is equivalent to the previously defined concept `Addable` and so is the following function template `add2` using the named concept `Addable`.

Use of the concept `Addable`

```
template<Addable T>
T add2(T a, T b) { return a + b; }
```

Concepts should encapsulate general ideas and give them a self-explanatory name for reuse. They are invaluable for maintaining code. Anonymous concepts read more like syntactic constraints the template parameters.

4.1.8.2.2 Type Requirements

In a type requirement, you have to use the keyword `typename` together with a type name.

The concept `TypeRequirement`

```
template<typename T>
concept TypeRequirement = requires {
    typename T::value_type;
    typename Other<T>;
};
```

The concept `TypeRequirement` requires that type `T` has a nested member `value_type`, and that the class template `Other` can be instantiated with `T`.

Let's try this out:

Use of the concepts `TypeRequirement`

```
1  #include <iostream>
2  #include <vector>
3
4  template <typename>
5  struct Other;
6
7  template <>
8  struct Other<std::vector<int>> {};
9
10 template<typename T>
11 concept TypeRequirement = requires {
12     typename T::value_type;
13     typename Other<T>;
14 };
15
16 int main() {
17
18     TypeRequirement auto myVec= std::vector<int>{1, 2, 3};
```

```
19
20  }
```

The expression `TypeRequirement auto myVec = std::vector<int>{1, 2, 3}` (line 18) is valid. A `std::vector`[20] has an inner member `value_type` (line 12) and the class template `Other` can be instantiated with `std::vector<int>` (line 13).

4.1.8.2.3 Compound Requirements

A compound requirement has the form

Compound requirement

```
{expression} noexcept(optional) return-type-requirement(optional);
```

In addition to a simple requirement, a compound requirement can have a `noexcept` specifier[21] and a requirement on its return type.

The concept `Equal`, demonstrated in the following example, uses compound requirements.

Definition and use of the concept `Equal`

```
1  // conceptsDefinitionEqual.cpp
2
3  #include <concepts>
4  #include <iostream>
5
6  template<typename T>
7  concept Equal = requires(T a, T b) {
8      { a == b } -> std::convertible_to<bool>;
9      { a != b } -> std::convertible_to<bool>;
10 };
11
12 bool areEqual(Equal auto a, Equal auto b){
13   return a == b;
```

[20] https://en.cppreference.com/w/cpp/container/vector
[21] https://en.cppreference.com/w/cpp/language/noexcept_spec

```
14 }
15
16 struct WithoutEqual{
17   bool operator==(const WithoutEqual& other) = delete;
18 };
19
20 struct WithoutUnequal{
21   bool operator!=(const WithoutUnequal& other) = delete;
22 };
23
24 int main() {
25
26     std::cout << std::boolalpha << '\n';
27     std::cout << "areEqual(1, 5): " << areEqual(1, 5) << '\n';
28
29     /*
30
31     bool res = areEqual(WithoutEqual(),  WithoutEqual());
32     bool res2 = areEqual(WithoutUnequal(),  WithoutUnequal());
33
34     */
35
36     std::cout << '\n';
37
38 }
```

The concept `Equal` (line 6) requires that its type parameter `T` supports the equal and not-equal operator. Additionally, both operators have to return a value that is convertible to a boolean. Of course, `int` supports the concept `Equal`, but this does not hold for the types `WithoutEqual` (line 16) and `WithoutUnequal` (line 20). Consequently, when I use the type `WithoutEqual` (line 31), I get the following error message when using the GCC compiler.

```
<source>:6:17:   in requirements with 'T a', 'T b' [with T = WithoutEqual]
<source>:7:9: note: the required expression '(a == b)' is invalid
    7 |     { a == b } -> std::convertible_to<bool>;
      |       ~~^~~~
<source>:8:9: note: the required expression '(a != b)' is invalid
    8 |     { a != b } -> std::convertible_to<bool>;
      |       ~~^~~~
```

`WithoutEqual` does not fulfill the concept `Equal`

4.1.8.2.4 Nested Requirements

A nested requirement has the form

Nested requirement

```
requires constraint-expression;
```

Nested requirements are used to specify requirements on type parameters.

Here is another way to define the concept `UnsignedIntegral` (see logical combinations of concepts and predicates):

The concepts `Integral`, `SignedIntegral`, and `UnsignedIntegral`

```
1  // nestedRequirements.cpp
2
3  #include <type_traits>
4
5  template <typename T>
6  concept Integral = std::is_integral<T>::value;
7
8  template <typename T>
9  concept SignedIntegral = Integral<T> && std::is_signed<T>::value;
10
11 // template <typename T>
12 // concept UnsignedIntegral = Integral<T> && !SignedIntegral<T>;
13
14 template <typename T>
15 concept UnsignedIntegral = Integral<T> &&
```

```
16 requires(T) {
17     requires !SignedIntegral<T>;
18 };
19
20 int main() {
21
22     UnsignedIntegral auto n = 5u;  // works
23     // UnsignedIntegral auto m = 5;   // compile time error, 5 is a sig\
24 ned literal
25
26 }
```

Line 14 uses with the concept `SignedIntegral` a nested requirement to refine the concept `Integral`. Honestly, the commented-out concept `UnsignedIntegral` in line 11 is more convenient to read.

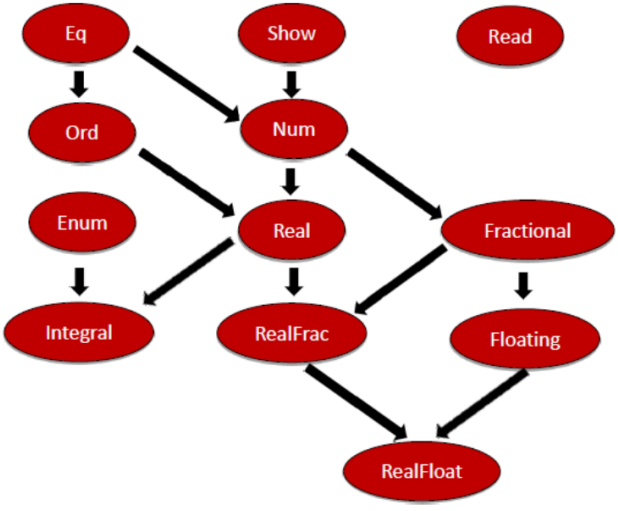
The concept `Ordering` in the following section demonstrates the use of nested requirements.

4.1.9 Application

In the previous sections I answered two essential questions about concepts: "How can a concept be used?" and "How can you define your concepts?". In this section, I want to apply the theoretical knowledge provided in those sections to define more advanced concepts such as `Ordering`, `SemiRegular`, and `Regular`.

4.1.9.1 The Concepts `Equal` and `Ordering`

I presented already in the short detour to the long, long history of concepts a part of Haskell's type classes hierarchy:



Haskell Type Classes Hierarchy

The class hierarchy shows that the type class `Ord` is a refinement of the type class `Eq`. Haskell expresses this elegantly.

A part of Haskell's type classes hierarchy

```
1  class Eq a where
2      (==) :: a -> a -> Bool
3      (/=) :: a -> a -> Bool
4
5  class Eq a => Ord a where
6      compare :: a -> a -> Ordering
7      (<) :: a -> a -> Bool
8      (<=) :: a -> a -> Bool
9      (>) :: a -> a -> Bool
10     (>=) :: a -> a -> Bool
11     max :: a -> a -> a
```

Each type `a` supporting the type class `Eq` (line 1), has to support equality (line 2) and inequality (line 3). Now to the interesting part of this definition. Each type `a` supporting the type class `Ord` has to support the type class `Eq` (`class Eq a => Ord a`

in line 5). Additionally, type `a` has to support the four comparison operators and the functions `compare` and `max` (lines 6 - 11).

Here is my challenge. Can we express Haskell's relationship between the type classes `Eq` and `Ord` with concepts in C++20? For simplicity, I ignore Haskell's functions `compare` and `max`.

4.1.9.1.1 The Concept `Ordering`

Thanks to the `requires` expression, the definition of the concept `Ordering` looks quite similar to the definition of the type class `ord` in Haskell.

The concept `Ordering`

```
template <typename T>
concept Ordering =
    Equal<T> &&
    requires(T a, T b) {
        { a <= b } -> std::convertible_to<bool>;
        { a < b } -> std::convertible_to<bool>;
        { a > b } -> std::convertible_to<bool>;
        { a >= b } -> std::convertible_to<bool>;
    };
```

The `Ordering` concept uses nested requirements under the hood. A type `T` supports the concept `Ordering` if it supports the concept `Equal` and, additionally, the four comparison operators. Let's try it out.

Definition and usage of the concept `Ordering`

```
// conceptsDefinitionOrdering.cpp

#include <concepts>
#include <iostream>
#include <unordered_set>

template<typename T>
concept Equal =
    requires(T a, T b) {
        { a == b } -> std::convertible_to<bool>;
        { a != b } -> std::convertible_to<bool>;
    };


template <typename T>
concept Ordering =
    Equal<T> &&
    requires(T a, T b) {
        { a <= b } -> std::convertible_to<bool>;
        { a < b } -> std::convertible_to<bool>;
        { a > b } -> std::convertible_to<bool>;
        { a >= b } -> std::convertible_to<bool>;
    };

template <Equal T>
bool areEqual(const T& a, const T& b) {
    return a == b;
}

template <Ordering T>
T getSmaller(const T& a, const T& b) {
    return (a < b) ? a : b;
}

int main() {
```

```
36
37      std::cout << std::boolalpha << '\n';
38
39      std::cout << "areEqual(1, 5): " << areEqual(1, 5) << '\n';
40
41      std::cout << "getSmaller(1, 5): " << getSmaller(1, 5) << '\n';
42
43      std::unordered_set<int> firSet{1, 2, 3, 4, 5};
44      std::unordered_set<int> secSet{5, 4, 3, 2, 1};
45
46      std::cout << "areEqual(firSet, secSet): " << areEqual(firSet, secSe\
47  t) << '\n';
48
49      // auto smallerSet = getSmaller(firSet, secSet);
50
51      std::cout << '\n';
52
53  }
```

The function template `areEqual` (line 25) requires that both arguments `a` and `b` have the same type and support the concept `Equal`. Additionally, the function template `getSmaller` (line 30) requires that both arguments support the concept `Ordering`. Of course, integrals such as `1` and `5` support both concepts. A `std::unordered_set`[22], as its name implies, does not fulfill the concept `Ordering`. Consequently, I commented out line 48.

```
areEqual(1, 5): false
getSmaller(1, 5): 1
areEqual(firSet, secSet): true
```

Use of the concept `Ordering`

Let's look at the more interesting case now. What happens, when we compile line 48: `auto smallerSet = getSmaller(firSet, secSet);`? The GCC compiler complains unambiguously that a `std::unordered_set` is not a valid argument for the function template `getSmaller`.

[22] https://en.cppreference.com/w/cpp/container/unordered_set

```
<source>:48:48:   required from here
<source>:16:9:   required for the satisfaction of 'Ordering<T>' [with T = std::unordered_set<int, std::hash<int>, std::equal_to<int>, std::allocator<int> >]
<source>:18:5:   in requirements with 'T a', 'T b' [with T = std::unordered_set<int, std::hash<int>, std::equal_to<int>, std::allocator<int> >]
<source>:19:13: note: the required expression '(a <= b)' is invalid
   19 |         { a <= b } -> std::convertible_to<bool>;
      |           ~~^~~~
<source>:20:13: note: the required expression '(a < b)' is invalid
   20 |         { a < b } -> std::convertible_to<bool>;
      |           ~~^~~
<source>:21:13: note: the required expression '(a > b)' is invalid
   21 |         { a > b } -> std::convertible_to<bool>;
      |           ~~^~~
<source>:22:13: note: the required expression '(a >= b)' is invalid
   22 |         { a >= b } -> std::convertible_to<bool>;
      |           ~~^~~~
```

Erroneous usage of the function template `getSmaller`

The `Ordering` concept is already part of the C++20 standard.

- `std::three_way_comparable`: is equivalent to the concept `Ordering` presented above
- `std::three_way_comparable_with`: allows the comparison of values of different types; e.g.: `1.0 < 1.0f`

With C++20, we get the three-way comparison operator, also known as the spaceship operator `<=>`. I present it in full depth in the three-way comparison operator chapter.

4.1.9.2 The Concepts `SemiRegular` and `Regular`

When you want to define a concrete type that works well in the C++ ecosystem, you should define a type that "behaves like an `int`". Formally, your concrete type should be a `regular` type. In this section, I define the concepts `SemiRegular` and `Regular`.

`SemiRegular` and `Regular` are essential ideas in C++. Sorry, I should say concepts. For example, here is rule T.46 from the C++ Core Guidelines: T.46: Require template arguments to be at least `Regular` or `SemiRegular`[23]. Now, only one important question is left to answer: What are `Regular` or `SemiRegular` types? Before I dive into the details, this is the informal answer:

- A `regular` type "behaves like an `int`." It can be copied and the result of the copy operation is independent of the original one and has the same value.

[23] http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Rt-regular

Okay, let me be more formal. A `regular` type is also a `semiregular` type, so let's begin.

`Regular` Types

Alexander Stepanov[24], the designer of the Standard Template Library, defined the terms `regular` type and `semiregular` type. A type, according to him, is `regular` if it supports these functions:

- Copy construction
- Assignment
- Equality
- Destruction
- Total ordering

Copy construction implies default construction and Equality implies Inequality. When Stepanov defined the requirements above, move semantics was not present in C++. The book Elements of Programming[25], which Alexander Stepanov wrote together with Paul McJones[26], is devoted to `regular` types.

4.1.9.2.1 The Concept `SemiRegular`

A `semiregular` type `X` has to support the Big Six and has to be swappable. The Big Six consists of the following functions:

- Default constructor: `X()`
- Copy constructor: `X(const X&)`
- Copy assignment: `X& operator = (const X&)`
- Move constructor: `X(X&&)`
- Move assignment: `X& operator = (X&&)`
- Destructor: `~X()`

[24] https://en.wikipedia.org/wiki/Alexander_Stepanov
[25] http://elementsofprogramming.com/
[26] https://www.mcjones.org/paul/

Additionally, `X` has to be swappable: `swap(X&, X&)`

Thanks to the type-traits library[27], defining the corresponding concept is a no-brainer. First, I define the type trait `isSemiRegular` and then use it to define the concept `SemiRegular`.

```
1  template<typename T>
2  struct isSemiRegular: std::integral_constant<bool,
3                                        std::is_default_constructible<T>:\
4  :value &&
5                                        std::is_copy_constructible<T>::va\
6  lue &&
7                                        std::is_copy_assignable<T>::value\
8   &&
9                                        std::is_move_constructible<T>::va\
10 lue &&
11                                       std::is_move_assignable<T>::value\
12  &&
13                                       std::is_destructible<T>::value &&
14                                       std::is_swappable<T>::value >{};
15
16
17 template<typename T>
18 concept SemiRegular = isSemiRegular<T>::value;
```

The type trait `isSemiRegular` (line 1) is fulfilled when all type traits to the Big Six (lines 3 - 8) and the type trait `std::is_swappable` (line 9) are fulfilled. The remaining step to define the concept `SemiRegular` is to use the type traits `isSemiRegular` (line 13).

Let's continue with the concept `Regular`.

4.1.9.2.2 The Concept `Regular`

There is only one step and we are done with defining the concept `Regular`. In addition to the requirements of the concept `SemiRegular`, the concept `Regular` requires that

[27] https://en.cppreference.com/w/cpp/header/type_traits

the type is equally comparable. I already defined the `Equal` concept in the section on requires expressions. Consequently, you are already done. You only have to conjunct the concepts `Equal` and `SemiRegular`.

Definition of the concept `Regular`

```
template<typename T>
concept Regular = Equal<T> &&
                  SemiRegular<T>;
```

Now, I'm curious. How can we define the corresponding concepts `std::semiregular` and `std::regular` in C++20?

4.1.9.2.3 `std::semiregular` and `std::regular`

C++20 combines the concepts `std::semiregular` and `std::regular` using of existing type traits and concepts.

Definition of the concept std::semiregular and std::regular

```
template<class T>
concept movable = is_object_v<T> && move_constructible<T> &&
                  assignable_from<T&, T> && swappable<T>;

template<class T>
concept copyable = copy_constructible<T> && movable<T> &&
                   assignable_from<T&, T&> &&
                   assignable_from<T&, const T&> && assignable_from<T&,\
 const T>;

template<class T>
concept semiregular = copyable<T> && default_initializable<T>;

template<class T>
concept regular = semiregular<T> && equality_comparable<T>;
```

Interestingly, the `std::regular` concept is defined similarly to concept `Regular`. On the other hand, the `std::semiregular` concept is combined with more elementary

concepts, such as `std::copyable` and `std::moveable`. The concept `std::movable` is based on the type-traits function `std::is_object`[28]. cppreference.com also provides a possible implementation of the compile-time predicate.

A possible implementation of the type trait std::is_object

```
template< class T>
struct is_object : std::integral_constant<bool,
                     std::is_scalar<T>::value ||
                     std::is_array<T>::value  ||
                     std::is_union<T>::value  ||
                     std::is_class<T>::value> {};
```

A type is an object if it is either a scalar, an array, a union, or a class.

To conclude this section, I want to apply the user-defined concept `Regular` and the C++20 concept `std::regular`. The program `regularSemiRegular.cpp` does this job.

Application of the concepts `Regular` and `SemiRegular`

```
1  // regularSemiRegular.cpp
2
3  #include <concepts>
4  #include <vector>
5  #include <type_traits>
6
7  template<typename T>
8  struct isSemiRegular: std::integral_constant<bool,
9                                      std::is_default_constructible<T>::\
10 :value &&
11                                     std::is_copy_constructible<T>::va\
12 lue &&
13                                     std::is_copy_assignable<T>::value\
14  &&
15                                     std::is_move_constructible<T>::va\
16 lue &&
17                                     std::is_move_assignable<T>::value\
```

[28] https://en.cppreference.com/w/cpp/types/is_object

```
 &&
                                        std::is_destructible<T>::value &&
                                        std::is_swappable<T>::value >{};

template<typename T>
concept SemiRegular = isSemiRegular<T>::value;

template<typename T>
concept Equal =
    requires(T a, T b) {
        { a == b } -> std::convertible_to<bool>;
        { a != b } -> std::convertible_to<bool>;
};

template<typename T>
concept Regular = Equal<T> &&
                  SemiRegular<T>;

template <Regular T>
void behavesLikeAnInt(T) {
    // ...
}

template <std::regular T>
void behavesLikeAnInt2(T) {
    // ...
}

struct EqualityComparable { };
bool operator == (EqualityComparable const&,
                  EqualityComparable const&) {
    return true;
}

struct NotEqualityComparable { };

```

```
54 int main() {
55
56     int myInt{};
57     behavesLikeAnInt(myInt);
58     behavesLikeAnInt2(myInt);
59
60     std::vector<int> myVec{};
61     behavesLikeAnInt(myVec);
62     behavesLikeAnInt2(myVec);
63
64     EqualityComparable equComp;
65     behavesLikeAnInt(equComp);
66     behavesLikeAnInt2(equComp);
67
68     NotEqualityComparable notEquComp;
69     behavesLikeAnInt(notEquComp);
70     behavesLikeAnInt2(notEquComp);
71
72 }
```

I put all pieces from the previous code-snippets together to define the concept `Regular` (line 27). The function templates `behavesLikeAnInt` (line 31) and `behavesLikeAnInt2` (line 36) check if the arguments "behave like an int." This means the user-defined concept `Regular` and the C++20 concept `std::regular` are used to establish the condition. As the name suggests, the type `EqualityComparable` (line 41) supports equality, but the type `NotEqualityComparable` (line 47) does not. The use of the type `NotEqualityComparable` in both function calls (lines 64 and 65) is the most interesting part of the program.

Although I'm in the early stage of concepts implementation, I want to compare the error messages of a new GCC and MSVC compilers.

- **GCC**

I used the current GCC 10.2 with the command line argument `-std=c++20` on Compiler Explorer[29]. These are essentially the error messages when I use the user-

[29] https://godbolt.org/

defined concept `Regular` (line 64):

```
<source>:23:13: note: the required expression '(a == b)' is invalid
   23 |         { a == b } -> std::convertible_to<bool>;
      |           ~~^~~~
<source>:24:13: note: the required expression '(a != b)' is invalid
   24 |         { a != b } -> std::convertible_to<bool>;
      |           ~~^~~~
```

Error message when using the concept `Regular`

The C++20 concept `std::regular` is more comprehensive. Consequently, the call in line 65 gives a more comprehensive error message:

```
/opt/compiler-explorer/gcc-10.2.0/include/c++/10.2.0/concepts:282:10: note: the required expression '(__t == __u)' is invalid
  282 |     { __t == __u } -> __boolean_testable;
      |       ~~~~^~~~~~
/opt/compiler-explorer/gcc-10.2.0/include/c++/10.2.0/concepts:283:10: note: the required expression '(__t != __u)' is invalid
  283 |     { __t != __u } -> __boolean_testable;
      |       ~~~~^~~~~~
/opt/compiler-explorer/gcc-10.2.0/include/c++/10.2.0/concepts:284:10: note: the required expression '(__u == __t)' is invalid
  284 |     { __u == __t } -> __boolean_testable;
      |       ~~~~^~~~~~
/opt/compiler-explorer/gcc-10.2.0/include/c++/10.2.0/concepts:285:10: note: the required expression '(__u != __t)' is invalid
  285 |     { __u != __t } -> __boolean_testable;
      |       ~~~~^~~~~~
```

Error message when using the concept std::regular

- MSVC

The error message given by the MSVC compiler is too unspecific.

```
x64 Native Tools Command Prompt for VS 2019

C:\Users\seminar>cl.exe /EHsc /std:c++latest regularSemiRegular.cpp
Microsoft (R) C/C++ Optimizing Compiler Version 19.27.29112 for x64
Copyright (C) Microsoft Corporation.  All rights reserved.

/std:c++latest is provided as a preview of language features from the latest C++
working draft, and we're eager to hear about bugs and suggestions for improvements.
However, note that these features are provided as-is without support, and subject
to changes or removal as the working draft evolves. See
https://go.microsoft.com/fwlink/?linkid=2045807 for details.

regularSemiRegular.cpp
regularSemiRegular.cpp(64): error C2672: 'behavesLikeAnInt': no matching overloaded function found
regularSemiRegular.cpp(64): error C7602: 'behavesLikeAnInt': the associated constraints are not satisfied
regularSemiRegular.cpp(32): note: see declaration of 'behavesLikeAnInt'
regularSemiRegular.cpp(65): error C2672: 'behavesLikeAnInt2': no matching overloaded function found
regularSemiRegular.cpp(65): error C7602: 'behavesLikeAnInt2': the associated constraints are not satisfied
regularSemiRegular.cpp(37): note: see declaration of 'behavesLikeAnInt2'

C:\Users\seminar>
```

Error message when using the concepts `Regular` and `std::regular`

As you can see from the screenshot, I applied version 19.27.29112 for x64 with the command line `/EHSC /std:c++latest`.

Concepts in C++20: An Evolution or a Revolution?

This small detour expresses my opinion. First, I present the facts, then I draw my conclusion. The facts are based on what has been presented in this chapter. So which arguments speak for evolution or for revolution?

Evolution

- Concepts promote working with generic code at a **higher level of abstraction**.
- Concepts give you **understandable error messages** when compiling a template fails. They provide nothing you could not achieve with the type-traits library[30], SFINAE[31], and `static_assert`[32].
- `auto` is a kind of unconstrained placeholder. With C++20, we can use concepts as **constrained placeholders**.
- With C++14, we could use **generic lambdas** as a convenient way to define function templates.

Revolution

- Concepts allow us, for the first time, to **verify template requirements**. Of course, you can also achieve the verification of template parameters with a combination of type-traits library[33], SFINAE[34], and `static_assert`[35], but this technique is way too advanced to regard it as a general solution.
- Thanks to the abbreviated function-templates syntax, defining templates has been radically improved.
- Concepts represent **semantic categories**, but not syntactic constraints. Instead of a concept such as `Addable`, which requires that a type supports the `+` operator, we should think in terms of a concept `Number`, where `Number` is a semantic category such as `Equal` or `Ordering`.

My Conclusion

There are many arguments whether concepts are an evolutionary step or a revolutionary jump. Mainly because of the semantic categories, I'm on the revolution side. Concepts such as `Number`, `Equality`, or `Ordering` remind me of Plato's[36] world of ideas. *It is revolutionary that we can now reason about programming in such categories.*

[30] https://en.cppreference.com/w/cpp/header/type_traits

Distilled Information

- Functions or classes defined on a specific type or a type parameter have their set of problems. Concepts overcome these problems by putting semantic constraints on type parameters.
- Concepts can be applied in requires clauses, in trailing requires clauses, as constrained template parameters, or in the abbreviated function templates.
- Concept are compile-time predicates that can be used for all kinds of templates. You can overload on concepts, specialize templates on concepts, use concepts for member functions or variadic templates.
- Thanks to C++20 and concepts, the use of unconstrained placeholders (`auto`) and constrained placeholders (concepts) is unified. Whenever you use `auto`, you can use concepts in C++20.
- Thanks to the new abbreviated function-templates syntax, defining a function template has become a piece of cake.
- Don't reinvent the wheel. Before you define your own concepts, study the rich set of predefined concepts in the C++20 standard. When you define your concepts, you can apply two techniques: combine concepts and compile-time predicates or use requires expressions.

[31] https://en.cppreference.com/w/cpp/language/sfinae
[32] https://en.cppreference.com/w/cpp/language/static_assert
[33] https://en.cppreference.com/w/cpp/header/type_traits
[34] https://en.cppreference.com/w/cpp/language/sfinae
[35] https://en.cppreference.com/w/cpp/language/static_assert
[36] https://en.wikipedia.org/wiki/Plato